C64 Visual Assembler — User Manual

Version 1.7.1

A visual, block-based 6502 assembler for the Commodore 64. Build programs by dragging and dropping instruction blocks, and see the generated assembly and machine code in real time.

Table of Contents

---

1. Interface Overview

The app is split into three main panels:

Panel	Description
Left — Palette	All available instruction and macro blocks. Search or browse by category.
Center — Program	Your program. Drag blocks here, reorder them, edit operands.
Right — Output	Live ASM view and/or memory monitor output.

---

2. Block Palette

The palette on the left lists all available blocks grouped by category:

- **Data movement** — LDA, LDX, STA, STX, ...
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, ...
- **Logic** — AND, ORA, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, ...
- **Register operations** — TAX, TAY, INX, DEX, ...
- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, ...
- **Illegal instructions** — LAX, SAX, DCP, ...
- **Structure** — LABEL, COMMENT, REGION, ENDREGION
- **Macros** — LOOP, NEXT, FOR, ENDF, PUSH, PULL, TEXT, BYTE, WORD, FILL, ALIGN, STRING, DATA, RAWBYTES, RAWTEXT, PETSCII, INCBIN, SID, INCLUDE, TABLE, ORG, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF, SPRITE_INIT, SPRITE_POS, WAIT_RASTER, JOYSTICK, MOUSE, SPRITE_COL, LOADFILE, REU_CHECK, REU_STASH, REU_FETCH, REU_SWAP, TURBO_SET, SUPERCPU_DETECT, TURBO_ENABLE

Use the **search box** at the top of the palette to filter by name. Click the **Add selected block** button or drag a block into the program area.

---

3. Program Area

- **Drag & drop** blocks from the palette, or **reorder** existing blocks by dragging their handle (≡).
- Each block shows its **mnemonic**, **operand field**, and **addressing mode selector** (where applicable).
- Click the **▸ / ▾** toggle to collapse or expand a block.
- Use the **× (delete)** button on a block to remove it.
- **Collapse All** button folds all blocks at once.

Operand input

- For branch/jump instructions (`BNE`, `JMP`, `JSR`, etc.) a **label picker** dropdown appears — click a defined label to insert it.
- Number format follows the **HEX / DEC** toggle in the toolbar (see section 5).

---

4. ASM View

The right panel shows the generated output in real time.

Output modes

Mode	Description
ASM	6502 assembly source with addresses and labels
Monitor	Hex / byte dump (C64 monitor style)
Disasm	Pure 6502 disassembly: address · hex bytes · mnemonics with resolved numeric operands. Macros are expanded to individual instructions (TEXT → LDA/STA pairs, LOOP → LDX, etc.). BYTE/WORD/FILL data shown as chunked hex dump. No macro names, comments, or annotations in output.
Both	ASM on top, monitor below
Options	Program settings panel — number format, macro source toggle, debugger params

Options tab

The **Options** tab contains the settings that affect code generation and output display:

- **Macro source** — when ON, macro definition blocks (MACRO...ENDM) show their source code inline in the ASM view.
- **Program start address** — now set via an **ORG block** in the program area rather than a separate input field. The first ORG block defines the program's load address; subsequent ORG blocks start additional sections at different addresses.

- **Debugger params** — three inline toggles controlling which flags are passed to the external debugger on launch:
  - **`-jmp` ON/OFF** — jump directly to the program's start address after loading.
  - **`-unpause` ON/OFF** — unpause the debugger immediately on load.
  - **`-wait` ms ON/OFF** — adds a `-wait <ms>` delay before unpausing; select 500 ms or 1000 ms from the dropdown.
- **Compile info** — shows a summary of the compiled program (code start address, size, BASIC SYS stub status).

Clicking an ASM line

Click any line in the ASM view to **highlight the corresponding block** in the program area.

ASM line numbers

The ASM panel displays **line numbers** (`001 |`, `002 |`, …) to make troubleshooting easier when a compile error points to a specific line.

- The visual line numbers are for diagnostics only.
- **Copy ASM** still copies the clean source text **without** line-number prefixes.

Compile progress modal

During heavier actions, a centered progress modal appears with a progress bar:

- **Run in VICE** — compiling/building PRG and launching emulator.
- **Debug** — compiling/building PRG and launching debugger.
- **Import ASM** — parsing and materializing blocks from pasted source.

The modal closes automatically when the action completes or fails.

---

5. Settings & Toolbar

Control	Description
Number base (HEX / DEC / BIN)	Sets the display/input format for operands throughout the UI. BIN mode displays values as binary with `%` prefix (e.g. `%11111000`). The ASM view always shows each block in its own format.
Language	Switch between English and Hungarian
Theme	Light / Dark / OLED — select from the theme picker in the Settings menu. OLED uses a pure-black background for AMOLED displays.
CRT retro mode	Toggles a full-screen CRT filter: scanlines, phosphor vignette, flicker, and barrel distortion. State is saved between sessions.
BASIC SYS stub	Prepends a BASIC line that calls SYS to your program's origin
Sample	Load a built-in example program
Zoom in / out	Scale the block UI (affects all block elements)

Control	Description
Save Project	Save the current program as a `.json` project file
Load Project	Load a previously saved project
Open Project (`Menu → File`)	Open a multi-file `.proj` project and open all source files as tabs
Save Project (`Menu → File`)	Save the current `.proj` project (project panel must be open)
Close Project (`Menu → File`)	Close the currently open project and all its file tabs. Prompts to save unsaved changes. The project panel resets to its empty state.
Import ASM	Opens a paste dialog and imports textual 6502 ASM into blocks
Save PRG	Export the compiled binary as a `.prg` file
Run (split button)	The main ▶ **Run** button runs the current mode; click the ▾ arrow to switch between: **Run as PRG** (compile and launch VICE directly), **Run via D64** (package into a .d64 disk image and launch VICE), or **Run on hardware** (send PRG to a C64 Ultimate / 1541 Ultimate device). See Section 12 and Section 13.
Debug (RetroDebugger)	Compile and launch in RetroDebugger with breakpoints, symbols, and autostart flags (see Section 9)
Hardware Settings	Open the hardware configuration dialog — configure VICE, RetroDebugger, and C64 Ultimate (host, password, connection test). See Section 13.
New program...	Opens a confirmation dialog, then clears all blocks from the program area
Collapse All	Collapse all blocks
About	Version info
What's New	Changelog
Knowledge Base	Reference links (6502 opcodes, C64 KERNAL, memory map, colors)
Check for Update	Open the itch.io page to check for a newer release

Import ASM (quick reference)

The Import dialog accepts common 6502 source patterns and converts them into blocks:

- `* = $1500` → ORG block
- `Label:` → LABEL block
- `Label: .byte 0` → LABEL + BYTE blocks
- `.byte ...` → BYTE block
- `; comment` (or inline `; ...`) → COMMENT block
- instructions (`lda`, `jsr`, `beq`, etc.) → instruction blocks with detected addressing mode

Import parsing notes and best practices

- Local labels like `.wait` are imported as standard labels (dot removed), and references are normalized accordingly.
- For `($zp),Y` / `($zp,X)` style addressing, use a concrete zero-page byte (`$FB`, `$FC`, etc.) for best compatibility.
- Avoid ambiguous short labels that look like hex (`cc1`, `dead`, `beef`) in branch contexts; prefer names like `loop_cc1`.
- If your program starts with data (`.byte`) before executable code, add an explicit entry jump (for example `JMP Start`) at the top.

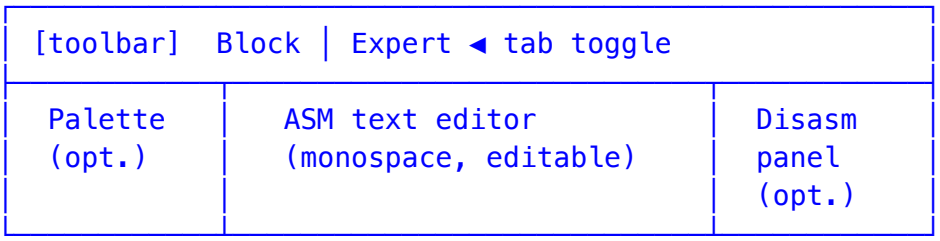
6. Expert Mode

Expert Mode is a full-featured direct-text 6502 assembly editor that lives alongside the block editor. Every tab can be in either Block mode or Expert mode — you switch between them freely at any time using the **Block / Expert** toggle in the top bar.

Switching modes

- **Block → Expert:** the current program is serialised to text (one instruction per line, labels, macros as directives). Edits in Expert mode are synced back to the block array whenever you switch back or trigger an action.
- **Expert → Block:** the text is parsed with `parseAsmText()` and the result replaces the block program. A compile-error dialog is shown if parsing fails.
- **Blank lines** are preserved through round-trips: empty lines in the Expert editor appear as thin dashed spacers in Block mode and are restored as empty lines when switching back to Expert.

Editor layout



Panel	Toggle	Description
Palette	`#expert-palette-btn`	The left block palette — drag blocks into the editor or click to insert at cursor
ASM editor	always visible	Full monospace textarea with live syntax highlight overlay
Disasm panel	`#expert-disasm-btn`	Pure 6502 disassembly: each instruction shows address, hex bytes, and numeric operands; macros fully expanded

Toolbar buttons

Button	ID	Function
Format	`#expert-format-btn`	Auto-format source (labels to col 0, 4-space indent, 1-space mnemonic/operand)
Load .asm	`#expert-load-asm-btn`	Open a `.asm` file — the content is loaded into a **new tab** with the filename as the tab label. Each loaded file becomes an independent tab with its own program blocks and editor state.
Save .asm	`#expert-save-asm-btn`	Save editor content to a `.asm` file (file dialog on first save)
Build Info	`#expert-build-info-btn`	Open the Build Info dialog (origin, size, labels, errors)
HL	`#expert-hl-btn`	Toggle syntax highlighting (disable for very large files)
Palette	`#expert-palette-btn`	Show/hide the left mnemonic palette
Disasm	`#expert-disasm-btn`	Show/hide the disassembly panel (pure 6502, macros expanded)
Monitor	`#expert-monitor-btn`	Show/hide the monitor hex-dump panel

Error highlighting

Lines that fail to compile are highlighted in **red** (tinted background + left accent border) in real time, 350 ms after each keystroke. The first error message is also shown in the status bar. Fix the line and the highlight disappears automatically.

Syntax highlight

The editor uses a transparent `<div>` overlay (`expert-hl`) that mirrors the textarea content with coloured `<span>` elements. Highlight can be toggled off with the **HL** button for performance on very large programs.

Colour	Token
Yellow-green	Mnemonics (`LDA`, `STA`, `JMP`, …)
Blue	Directives (`.byte`, `.word`, `.fill`, `*=`, …)
Orange	Numbers (`$FF`, `%1010`, `255`)
Cyan	Labels (lines ending in `:`)
Teal	String literals
Dark green	Comments (`; …`)

Source formatter

Click the **Format** button (`#expert-format-btn`) to auto-format the current source:

- Label definitions are moved to column 0.
- Instructions are indented with 4 spaces.
- Mnemonics are uppercased.
- Exactly one space between mnemonic and operand (extra whitespace is normalised).
- If the source is already formatted, a `"Already formatted"` status is shown.

Project panel & tabs

Expert mode supports a **project panel** (`#expert-project-panel`) for multi-file `.proj` projects:

- A `.proj` file is a JSON manifest that lists source files and their metadata.
- Open a project with **Menu → File → Open project** or drag a `.proj` file onto the window.
- Each file in the project opens as a separate **tab** in the tab bar at the top of the editor.
- **Close Project** (`Menu → File → Close project` / `#menu-close-project`) closes the current project and all its file tabs at once. Prompts to save any unsaved changes before closing. The project panel resets to its empty state and `_expertProjectData` is cleared.
- Each file can be marked as the **startup file** (★ star icon). When a startup file is set, the **Run** button (PRG, D64, Ultimate) always assembles and runs that file's code — regardless of which tab is currently active. This works in both block mode and Expert mode.

Tab bar

The tab bar appears above the editor when there is more than one tab open.

Feature	Description
Dirty dot	A small accent-coloured dot on the tab name indicates unsaved changes
Scroll arrows	Left/right scroll buttons appear when there are more tabs than fit the bar
Close (×)	Closes the tab; prompts to save if the tab is dirty
File extension	The full filename including extension (`.c64va`, `.json`) is shown

> **Tip:** Palette sync (`#expert-palette-sync-btn`) keeps the palette selection in sync with the mnemonic at the cursor. Disable it when you prefer not to have the palette jump around as you edit.

---

7. Addressing Modes

Each 6502 instruction supports one or more addressing modes. The mode selector appears on each block.

Mode	Label	Example	Description
implied	Implied	`NOP`	No operand; the instruction is self-contained

Mode	Label	Example	Description
immediate	Immediate	`LDA #$FF`	Inline constant; the assembler adds # automatically
zeroPage	Zero page	`LDA $10`	Single byte address in page zero (0–255)
zeroPageX	Zero page,X	`LDA $10,X`	Zero page address + X register offset (result wraps in page 0)
zeroPageY	Zero page,Y	`LDX $FB,Y`	Zero page address + Y register offset
absolute	Absolute	`LDA $0400`	Full 16-bit memory address
absoluteX	Absolute,X	`LDA $0400,X`	16-bit address + X register offset
absoluteY	Absolute,Y	`LDA $0400,Y`	16-bit address + Y register offset
relative	Relative/Label	`BNE loop`	For branch instructions; enter a label name or target address
indirectX	Indirect,X	`LDA ($FB,X)`	Zero page indexed indirect (operand = zero page address, 1 byte)
indirectY	Indirect,Y	`LDA ($FB),Y`	Zero page indirect indexed (operand = zero page address, 1 byte)
indirect	Indirect	`JMP ($0100)`	Indirect; only usable with JMP

Label expressions as operands

Any operand field that accepts an address or immediate value also accepts a **constant name** (from a `CONST` block or a `LABEL`) directly. Additionally, you can use **label+offset** or **label−offset** expressions to reference an address relative to a named constant:

Syntax	Example	Description
`label`	`STA screen_ram,X`	Resolves to the label/constant value
`label+$hex`	`STA screen_ram+$0100,X`	Label address plus a hex offset
`label+decimal`	`STA screen_ram+256,X`	Label address plus a decimal offset
`label-$hex`	`LDA table-$10`	Label address minus a hex offset
`#<label`	`LDA #<screen_ram`	Low byte of the label address
`#>label`	`LDA #>screen_ram`	High byte of the label address

Syntax	Example	Description
`*`	`BNE *`	Current program counter (the instruction's own address); branches with `*` generate an infinite self-loop (offset `$FE`)

Example — clear two screen pages using a CONST:

```
; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear
```

---

8. Standard 6502 Instructions

Data Movement

Mnemonic	Description	Modes
`LDA`	Load Accumulator	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`LDX`	Load X register	immediate, zeroPage, zeroPageY, absolute, absoluteY
`LDY`	Load Y register	immediate, zeroPage, absolute, absoluteX
`STA`	Store Accumulator	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`STX`	Store X register	zeroPage, zeroPageY, absolute
`STY`	Store Y register	zeroPage, absolute

Arithmetic

Mnemonic	Description	Notes
`ADC`	Add with carry	Set carry with `SEC` before use in most cases
`SBC`	Subtract with carry	Set carry with `SEC` before subtraction
`INC`	Increment memory	—
`DEC`	Decrement memory	—
`CMP`	Compare with A	Sets flags; does not modify A
`CPX`	Compare with X	—

Mnemonic	Description	Notes
`CPY`	Compare with Y	—

Logic

Mnemonic	Description
`AND`	Logical AND with Accumulator
`ORA`	Logical OR with Accumulator
`EOR`	Exclusive OR with Accumulator
`BIT`	Test bits in memory against A (sets N, V, Z flags)

Jumps & Branches

Mnemonic	Description
`JMP`	Unconditional jump (absolute or indirect)
`JSR`	Jump to subroutine (saves return address on stack)
`RTS`	Return from subroutine
`RTI`	Return from interrupt
`BNE`	Branch if Not Equal (Z=0)
`BEQ`	Branch if Equal (Z=1)
`BCC`	Branch if Carry Clear (C=0)
`BCS`	Branch if Carry Set (C=1)
`BMI`	Branch if Minus (N=1)
`BPL`	Branch if Plus (N=0)
`BVC`	Branch if Overflow Clear (V=0)
`BVS`	Branch if Overflow Set (V=1)

Register Operations

Mnemonic	Description
`TAX`	Transfer A → X
`TAY`	Transfer A → Y
`TXA`	Transfer X → A
`TYA`	Transfer Y → A
`TSX`	Transfer Stack pointer → X

Mnemonic	Description
TXS	Transfer X → Stack pointer
INX	Increment X
DEX	Decrement X
INY	Increment Y
DEY	Decrement Y

Shift & Rotate

Mnemonic	Description
ASL	Arithmetic Shift Left
LSR	Logical Shift Right
ROL	Rotate Left through Carry
ROR	Rotate Right through Carry

Stack

Mnemonic	Description
PHA	Push Accumulator onto stack
PHP	Push Processor status onto stack
PLA	Pull Accumulator from stack
PLP	Pull Processor status from stack

System / Flags

Mnemonic	Description
CLC	Clear Carry flag
CLD	Clear Decimal mode
CLI	Clear Interrupt disable
CLV	Clear Overflow flag
SEC	Set Carry flag
SED	Set Decimal mode
SEI	Set Interrupt disable
NOP	No operation
BRK	Force break / software interrupt

Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

`LAX`, `SAX`, `DCP`, `ISC`, `SLO`, `RLA`, `SRE`, `RRA`, `ANC`, `ALR`, `ARR`, `AXS`

---

9. Macro Blocks — Reference

Macro blocks generate multiple instructions or data directives automatically. They are found in the **Macros** category of the palette.

LABEL

Creates a named label that can be used as a jump target.

Field	Description
Label name	Identifier used in `JMP`, `JSR`, `BNE`, etc.

Generated ASM:

```
loop:  ; $0820
```

The address is shown as a comment. Labels have **0 byte size**.

COMMENT

Inserts a comment line in the ASM output. No bytes generated.

Generated ASM:

```
; Your comment text here
```

BYTE

Inserts an arbitrary sequence of raw bytes.

Field	Description
Operand	Comma-separated byte values (e.g. `$01, $02, $FF` or `1, 2, 255`)

Generated ASM:

```
    .byte $01, $02, $FF
```

Size: Number of bytes in the list.

WORD

Inserts 16-bit values stored as little-endian LO/HI byte pairs.

Field	Description
Operand	Comma-separated 16-bit values (e.g. `$0400, $C000`)

Generated ASM:

```
    .word $0400, $C000
```

Size: 2 bytes per word.

FILL

Generates a repeated sequence of the same byte value.

Field	Description
Operand	`count,value` — e.g. `256,0` fills 256 bytes with zero

Generated ASM:

```
    .fill 256, $00
```

Size: The count value in bytes.

ALIGN

Advances the program counter to the next boundary of the given size. Inserts padding bytes as needed.

Field	Description
Boundary	Alignment value — e.g. `64` (sprite boundary), `256` (page), `$2000` (bitmap)

Generated ASM:

```
    ; ALIGN 64 → $0840 (12 bytes padding)
```

Size: Dynamic — depends on the current program counter position.

> **Tip:** Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

TEXT

Writes a text string to the C64 screen RAM at a given row/column position by generating direct LDA/STA pairs targeting the screen RAM at `$0400`.

Field	Description
Text	The string to display
X	Column (0–39)
Y	Row (0–24)
Label (optional)	Assigns a label pointing to the computed screen address

Case auto-detection: If the input contains any alphabetic character (`/[A-Za-z]/`), the macro uses lowercase charset screen codes: `a–z` → 1–26, `A–Z` → 65–90. This produces correct mixed-case output when the C64 charset is switched to lowercase mode (`$D018=$17`, e.g. `LDA #$17` / `STA $D018`). Pure numbers and symbols use standard screen codes for backward compatibility.

Generated ASM:

```
LDA #$08      ; 'H' (screen code, upper-case in standard charset)
STA $0400
LDA #$05      ; 'E' (screen code)
STA $0401
...
```

Characters are encoded as **screen codes** (not PETSCII). **Size:** `text.length × 5` bytes (LDA + STA per character).

STRING

Encodes a text string as **screen codes** and writes it to a given memory address using LDA/STA instruction pairs at runtime. Same case auto-detection as TEXT.

Field	Description
Text	The string to write
Address	Target memory address (e.g. `$C000`)
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)

Generated ASM:

```
LDA #$08      ; 'H' screen code
STA $C000
LDA #$05      ; 'E' screen code
```

```
    STA $C001
    ...
```

Each character is converted to a **screen code** (not PETSCII). The optional **Shift** value is added to every byte (`& $FF`). **Size:** `text.length × 5` bytes (one LDA + one STA per character).

DATA

Writes raw bytes to a given memory address using LDA/STA instructions at runtime.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address

Generated ASM:

```
    LDA #$01
    STA $C000
    LDA #$02
    STA $C001
    ...
```

Size: `byte_count × 5` bytes (one LDA + one STA per byte).

RAWBYTES

Places raw bytes at a given memory address — no runtime code is generated. The bytes appear in the deferred data section of the output.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address (e.g. `$C000`)
Label (optional)	Assigns a label pointing to the target address

Size in code: 0 bytes. The data is placed at the given address in the output.

> **Use this instead of DATA** when you don't need runtime initialization code and just want bytes at an address.

RAWTEXT

Encodes a text string as **screen codes** and places the bytes directly at a given memory address — no runtime code is generated. The data appears in the deferred section of the output. Same case auto-detection as TEXT and STRING.

Field	Description
Text	String to encode
Address	Target memory address
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)

Generated ASM:

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F   ; H E L L O (screen codes)
```

Size in code: 0 bytes. The data is placed at the given address in the output.

> **Difference from STRING:** RAWTEXT places the data as static bytes — no LDA/STA code is generated. The bytes are present in memory as soon as the PRG is loaded, before any code runs.

PETSCII

Places a text string encoded as PETSCII bytes at a given memory address. No runtime code is generated — the bytes are placed directly at the target address, similar to RAWBYTES.

Each printable ASCII character (codes 32–126) is stored using its standard ASCII value, which corresponds to the PETSCII uppercase range. The result is compatible with KERNAL CHROUT (`$FFD2`).

Field	Description
Text	String to encode as PETSCII bytes
Address	Target memory address (e.g. `$C000`)

Generated ASM:

```
; .petscii "HELLO" -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O
```

Size in code: 0 bytes. The data is placed at the target address as a deferred data section (like RAWBYTES).

Null terminator: Check the *"Append `$00` (null terminator)"* checkbox in the PETSCII block to automatically add a `$00` byte after the text. This is ideal for null-terminated string loops:

```
    LDX #$00
for0:
```

```
    LDA msg,X
    BEQ done        ; $00 stops the loop
    JSR $FFD2
    INX
    CPX #$20
    BNE for0
done:
    RTS
```

In expert mode, add `, null` after the text: `.petscii $C000, "HELLO", null`

Encoding rules:

Input	Byte value
Printable ASCII (space, letters, digits, punctuation)	Standard ASCII code (32–126)
Newline	`$0D` (RETURN)
Everything else	`$20` (space)

> **Tip:** Use PETSCII for data that will be output via CHROUT (`$FFD2`). For screen-code strings (different mapping), use the STRING macro instead.

INCBIN

Includes an external binary file and places its content at a given memory address.

Field	Description
File	Browse to select a `.bin`, `.prg`, `.sid`, or `.raw` file
Address	Target load address (e.g. `$C000`)

Generated ASM comment:

```
    ; INCBIN "music.bin" @ $C000 (2048 bytes)
    .byte $01, $02, ...
```

Size in code: 0 bytes (deferred data section). The binary is embedded at the given address.

SID

Loads a SID music file and embeds its raw data directly into the assembled PRG at a configurable memory address. Header metadata (Load/Init/Play addresses, title, author) is extracted automatically.

Field	Description
File	Browse to select a `.sid` file

Field	Description
Custom address (optional)	Override the SID's native load address (e.g. `$1000`). Leave empty to use the address from the SID header.

The block displays:

- **Title / Author** from the SID header
- **Load address** — where the data is placed in memory (effective address after any override)
- **Init address** — call this with JSR to initialize the music (adjusted for relocation if a custom address is used)
- **Play address** — call this with JSR on every frame in an IRQ handler (adjusted for relocation)
- A **(relocated)** badge appears when a custom address shifts the data from its original position

Generated ASM comment:

```
; SID "Ikari_Warriors.sid" @ $1000  Init:$1000  Play:$1006  (4096 bytes)
```

Size in code: 0 bytes inline. The SID binary is placed at the specified address as a deferred chunk in the PRG.

> **Important:** Most SID files contain hardcoded internal absolute addresses. They can only be relocated if the entire binary is shifted by the same offset. If a SID has internal jumps to `$10xx`, it must remain at `$1000` — moving it to a different address will break those internal references.

> **Typical usage:** Place an ORG block before the SID block to set its address. Call Init once at startup, then call Play every frame from a raster IRQ handler.

INCLUDE

Includes another Visual Assembler project file and expands its blocks inline at this position. The included blocks are read-only.

Field	Description
File	Browse to select a `.json` Visual Assembler project
Load address (optional)	If set (hex, e.g. `C000`), the included blocks are placed at that address — a synthetic `ORG` is injected before them, overriding any ORG block inside the included file. Leave empty to let the included file's own ORG blocks control placement.

Generated ASM (no address override):

```
; .include "library.json" — 12 block(s)
... (expanded blocks follow)
```

Generated ASM (with load address `C000`):

```
; .include "library.json" @ $C000 — 12 block(s)
*=$C000
... (expanded blocks follow)
```

> **Tip:** Use INCLUDE to build reusable subroutine libraries that you can share across projects. Set a load address when the library has no ORG of its own, or when you want to override its default placement.

TABLE

Defines a lookup table at a fixed memory address with a named label.

Field	Description
Name	Label identifier for the table (e.g. `color_table`)
Address	Fixed address where the table starts (e.g. `$C000`)

Generated ASM:

```
color_table:
```

The program counter jumps to the specified address for subsequent blocks. Place BYTE/WORD/FILL blocks after TABLE to fill the table content.

Size: 0 bytes.

ORG

Sets a new **program counter origin** — equivalent to the `*= $ADDR` assembler directive. The first ORG block in your program defines its primary load address. Use additional ORG blocks to split the program into multiple sections that load at different addresses.

Field	Description
Address	The new origin address (e.g. `0801` in HEX, or `2049` in DEC)
HEX / DEC	Toggle the address input between hexadecimal and decimal display

Generated ASM:

```
* = $C000
```

Size: 0 bytes. The ORG block itself generates no machine code.

Each ORG block starts a new code section. All blocks following an ORG (until the next ORG or end of program) are assembled relative to that address. When the PRG is exported, all sections are merged into

a single flat buffer: the load address is the lowest section start across all sections; gaps between sections are zero-filled.

Example — code at `$0801`, data table at `$C000`:

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

> **Tip:** Every program must start with an ORG block. The typical starting address for a C64 BASIC-loadable program is `$0801` (2049 decimal). When **BASIC SYS stub** is enabled, the assembler adds a short BASIC line at `$0801` and your code starts at `$080D`.

LOOP / NEXT

A counted loop pair using a register as the counter.

LOOP

Field	Description
Register	`X` or `Y` — the counter register
Count	Loop iteration count (hex or decimal, e.g. `0A` = 10)
Label	Auto-generated loop label (e.g. `loop0`)

Generated ASM:

```
    LDX #$0A
loop0:
```

Size: 2 bytes (LD_ opcode + immediate operand).

NEXT

Field	Description
Register	Automatically matched to the LOOP register
Label	Automatically linked to the LOOP label

Generated ASM:

```
    DEX
    BNE loop0
```

Size: 3 bytes (DEX + BNE + branch offset).

Example — clear 10 screen cells:

```
    LDX #$0A
loop0:
    LDA #$20        ; space character
    STA $0400,X
    DEX
    BNE loop0
```

FOR / ENDF

Forward counting loop pair. Unlike LOOP/NEXT which counts **down** (N→1), FOR/ENDF counts **up** (0→N-1) — ideal for string printing and array traversal.

FOR

Field	Description
Register	X or Y — the counter register
Count	Loop limit (hex or decimal, e.g. `$12` = 18). X/Y runs from 0 up to limit-1.
Label	Auto-generated loop label (e.g. `for0`)

Generated ASM:

```
    LDX #$00
for0:
```

Size: 2 bytes (LD_ opcode + `#$00`).

ENDF

Field	Description
Register	Automatically matched to the FOR register
Label	Automatically linked to the FOR label
Count	Automatically copied from the paired FOR

Generated ASM:

```
    INX
    CPX #$12
    BNE for0
```

Size: 5 bytes (IN_ + CP_ #imm + BNE offset).

Example — print a null-terminated string:

```
    LDX #$00
for0:
    LDA msg,X       ; msg = PETSCII string at fixed address
    BEQ done        ; null terminator → exit
    JSR $FFD2       ; CHROUT
    INX
    CPX #$12        ; 18 characters max
    BNE for0
done:
    RTS
```

> **LOOP vs FOR:** LOOP is better for known-iteration loops (delay, memory fill, pixel loops). FOR is better for string/array operations where you need a forward index (0→N). Both can use X or Y.

PUSH / PULL

Save and restore registers using the stack.

PUSH

Pushes one or more registers onto the stack. The order is always A → X → Y (innermost first).

Field	Description
Registers	Any combination: A, X, Y, AX, AY, XY, AXY

Generated ASM (example: AX):

```
    PHA
    TXA
    PHA
```

Size: 1 byte for A (PHA), 2 bytes for X or Y (transfer + push).

PULL

Restores registers from the stack in reverse order (Y → X → A).

Field	Description
Registers	Same as PUSH — must match the corresponding PUSH block

Generated ASM (example: `AX`):

```
    PLA
    TAX
    PLA
```

> **Important:** Always pair PUSH and PULL with the same register set, and in reverse order. `PUSH AX` must be matched by `PULL AX` (which internally does Y-first, then X, then A).

MACRO / ENDM / INVOKE

Define and reuse your own named code blocks, optionally with parameters.

MACRO (definition start)

Field	Description
Name	Identifier for the macro (e.g. `setColor`)
Params	Optional comma-separated parameter names (e.g. `color` or `color, count`)

Marks the beginning of a macro definition. All blocks between MACRO and ENDM become the macro body. The definition does **not** generate code where it appears.

Inside the body, use `{paramName}` as a placeholder wherever the argument value should be substituted at invoke time.

Generated ASM:

```
; .MACRO setColor (color)
    ... (body blocks)
; .ENDM
```

Expert mode syntax:

```
.macro setColor color
    LDA {color}
    STA $D020
.endm
```

ENDM (definition end)

Closes the current macro definition. No fields.

INVOKE

Inserts the contents of a named macro at this position, substituting any argument values for `{paramName}` placeholders in the body.

Field	Description
Macro name	Select from the dropdown of defined macros
Arguments	Comma-separated argument values matching the macro's parameter list (e.g. `#$07`)

Generated ASM:

```
; .invoke setColor(#$07)
    LDA #$07
    STA $D020
```

Expert mode syntax:

```
; single argument:
.invoke setColor(#$07)

; multiple arguments:
.invoke drawPixel($10, $20)

; no arguments:
.invoke clearScreen
```

The macro body is expanded inline — `{paramName}` placeholders are replaced with the supplied argument values. All addresses and labels are resolved in context. The space-separated form (`.invoke setColor #$07`) is also accepted for backwards compatibility.

> **Tip:** Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times with different arguments.

REGION / ENDREGION

Groups a set of blocks into a **named, collapsible section**. REGION and ENDREGION are purely visual — they generate **zero bytes** and have no effect on the assembled output.

Field	Description
Region name	Free-text label for the section (e.g. `init`, `game_loop`, `sprite_setup`)

Controls on the REGION block header (always visible):

- **▸ / ▾ toggle** — collapses or expands the entire region. When collapsed, all blocks between REGION and ENDREGION are hidden.

- ↕ **Expand all** — un-collapses every individually collapsed block inside the region and expands the region itself if needed.
- ⊖ **Select in ASM** — highlights the entire region's code range in the ASM view (from `; ===[ name ]===` to `; ===[/name]===`) and scrolls to it. Switches to the ASM tab automatically if it is not currently visible.
- ⧉ **Copy region** — copies the REGION block, all child blocks, and the matching ENDREGION into a clipboard. A ✓ flash confirms the copy.
- ⎘ **Paste region** — inserts the copied region as a new region immediately after the current region's ENDREGION and scrolls to it. The button is dimmed until a region has been copied.

Generated ASM:

```
; region init
    SEI
    LDA #$00
    STA $D020
; endregion init
```

Size: 0 bytes for both REGION and ENDREGION.

Example workflow:

1. Add a `REGION` block, set region name to `init`.
2. Add your initialization instructions below it.
3. Add an `ENDREGION` block to close the section.
4. Click ▸ on the REGION to collapse the whole section into one line while working on other parts of the program.

> **Note:** Regions **can be nested** — a REGION placed inside another region acts as a child group (syntax sugar for visual organisation). Each ENDREGION closes the nearest preceding unclosed REGION. Nesting has no effect on the assembled output.

DEFINE / IF / ELSE / ENDIF

Conditional assembly blocks. Add a `DEFINE` block to activate a named symbol — any `IF` block whose condition matches a `DEFINE` present in the program is assembled; its `ELSE` branch (if any) is skipped, and vice versa.

DEFINE

Field	Description
Symbol	One or more comma-separated identifiers to activate (e.g. `DEBUG` or `DEBUG, PAL`)

Generated ASM:

```
; .DEFINE DEBUG
```

```
; .DEFINE DEBUG, PAL
```

A single `DEFINE` block can activate multiple symbols at once using a comma-separated list. Place `DEFINE` blocks anywhere in the program (typically at the top). Removing the block deactivates all its symbols instantly.

IF

Field	Description
Condition	Identifier to test (must match a `DEFINE` symbol to be active)

Generated ASM:

```
; .IF DEBUG
```

Blocks between `IF` and `ENDIF` (or `ELSE`) are included or skipped based on whether the condition symbol has a matching `DEFINE` block in the program. Skipped blocks appear as `; [IF skipped] …` comments and generate **zero bytes**.

ELSE

No fields. Marks the alternative branch — assembled when the `IF` condition is *not* active.

Generated ASM:

```
; .ELSE
```

ENDIF

No fields. Closes the conditional block.

Generated ASM:

```
; .ENDIF
```

Size: 0 bytes for all four blocks. Only the content *between* them counts.

Example — debug border flash, release build skips it:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; red border
    STA $D020
; .ELSE
    LDA #$00        ; black (release)
```

```
    STA $D020
; .ENDIF
```

Example — multiple symbols in one DEFINE block:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL timing constant
; .ELSE
    LDA #$xx        ; NTSC timing constant
; .ENDIF
```

Nested `IF` blocks are supported. The innermost condition is evaluated independently; an outer skipped block causes all inner blocks to be skipped regardless of their condition.

CONST

Declares a named constant. The constant is added to the label table and can be referenced as an operand in any instruction block (LDA, STA, JSR, JMP, etc.).

Field	Description
Name	Identifier for the constant (e.g. `SCREEN`)
Value	Numeric value in the selected base (e.g. `0400` in HEX = address $0400)
Format	HEX or DEC — controls how the value is entered and displayed

Generated ASM:

```
; .CONST SCREEN = $0400
```

The constant name appears in the **label picker** dropdown of instruction blocks that support absolute/immediate addressing — simply click it to insert the constant name as the operand.

Size: 0 bytes.

SPRITE_INIT

Initialises a VIC-II sprite in a single block: sets the **data pointer**, **enable bit**, and **colour**.

Field	Description
Sprite #	Sprite number 0–7
Colour	Colour index 0–15 (C64 palette)
Data page	Sprite data address / 64 (e.g. `$21` if data is at `$0840`)

Generated ASM:

```
    LDA #$21
    STA $07F8       ; sprite pointer register ($07F8 + N)
    LDA $D015
    ORA #$01        ; set enable bit for sprite 0
    STA $D015
    LDA #$07
    STA $D027       ; sprite 0 colour register
```

Size: 18 bytes.

> **Sprite data page:** `data_address / 64`. With the default BASIC SYS stub, an `ALIGN 64` block after `JMP main` places sprite data at `$0840` → page = `$0840 / 64 = 33 = $21`.

SPRITE_POS

Sets a sprite's **static start position** (compile-time constant). Use this for initial placement; for animation use `INC`/`DEC` on the sprite's register directly.

Field	Description
Sprite #	Sprite number 0–7
X	Horizontal position 0–319
Y	Vertical position 0–255

Generated ASM (example: sprite 0, X=152, Y=100):

```
    LDA #$98        ; X low byte
    STA $D000       ; sprite 0 X register
    LDA $D010
    AND #$FE        ; clear X MSB for sprite 0 (X ≤ 255)
    STA $D010
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y register
```

For X > 255 the macro sets the corresponding bit in `$D010` instead of clearing it.

Size: 18 bytes.

> **Note:** `SPRITE_POS` bakes the position into the code at assemble time (`LDA #$xx`). To move a sprite at runtime use `INC $D000` / `DEC $D000` directly — see the `sprite-macro-demo` sample.

WAIT_RASTER

Waits for a specific VIC-II raster line — entirely **inline**, no JSR and no external label required.

Field	Description
Raster line	Target raster line in hex (e.g. `FF` = line 255)

Generated ASM:

```
wait:
    LDA $D012       ; current raster line
    CMP #$FF        ; target line
    BNE wait        ; loop back (-7 bytes)
```

Size: 7 bytes (the `BNE` offset `$F9` = −7 always points back to the `LDA`).

> **Tip:** Place `WAIT_RASTER` at the top of your game loop to synchronise with the display and prevent sprite tearing.

JOYSTICK

Reads a CIA joystick port and moves a sprite according to the direction pressed. Entirely **inline** — no JSR or label needed.

Field	Description
Port	`1` = port 1 (`$DC01`) or `2` = port 2 (`$DC00`)
Sprite #	Sprite number 0–7 (controls which X/Y register pair is updated)

Generated ASM (port 2, sprite 0):

```
    LDA $DC00       ; read CIA port 2
    LSR             ; bit 0 → carry (Up)
    BCS skip_up     ; carry set = NOT pressed
    DEC $D001       ; Y-1 (move up)
skip_up:
    LSR             ; bit 1 → carry (Down)
    BCS skip_down
    INC $D001       ; Y+1 (move down)
skip_down:
    LSR             ; bit 2 → carry (Left)
    BCS skip_left
    DEC $D000       ; X-1 (move left)
skip_left:
    LSR             ; bit 3 → carry (Right)
    BCS skip_right
    INC $D000       ; X+1 (move right)
skip_right:
```

Joystick bit map (active-LOW — bit = 0 means pressed):

Bit	Direction	CIA register
0	Up	$DC00 (port 2) / $DC01 (port 1)
1	Down	
2	Left	
3	Right	
4	Fire	(not handled by this macro)

Size: 27 bytes. The `BCS` offset is always `+3` (skips the following 3-byte `DEC`/`INC abs` instruction).

> **Typical usage:** Place inside a `gameloop` label with `WAIT_RASTER` first:
>
> ```
> gameloop:
>     WAIT_RASTER ($FF)
>     JOYSTICK (port=2, sprite=0)
>     JMP gameloop
> ```

MOUSE

Reads a Commodore 1351 proportional mouse via the SID chip's paddle inputs (`POTX` = `$D419`, `POTY` = `$D41A`) and moves a sprite proportionally. Entirely **inline** — no JSR or label needed. After selecting the control port on CIA1 `$DC00`, the macro waits roughly one SID conversion window (just over 512 machine cycles) before reading the POT registers, otherwise the reads can be stale or mid-switch. The movement decoding follows the standard 1351 driver pattern from Codebase64: it works from the raw POT samples, masks the delta to 7 bits, treats values `0..63` as positive and `64..127` as negative movement, then halves the delta (`LSR` / `ROR`) before applying it. On X, the sprite low byte is updated first and the matching `$D010` bit is toggled with the classic `TXA / ADC #$00 / AND #$01 / EOR $D010` pattern when a page crossing occurs. On Y, the decoded delta is inverted with `EOR #$FF` + `SEC` before `ADC`, so mouse-up moves the sprite upward in VICE as expected. The current inline implementation is 142 bytes long.

This implementation intentionally stays close to the standard 1351 routines instead of layering custom clamps and heuristics on top. That keeps the behavior more predictable, although very large motion between polls can still alias because the 1351 is sampled periodically rather than continuously. If you need smoother fast motion in a sample, poll the macro more than once per frame rather than adding more clipping logic.

Field	Description
Port	`1` = CIA `$DC00` bits `7:6` set to `%01`, `2` = CIA `$DC00` bits `7:6` set to `%10`
Sprite #	Sprite number 0–7 (controls which `$D000`/`$D001` pair is updated)
ZP byte X	Zero-page address (hex, 1 byte) used to store the previous POTX value (e.g. `FD`)
ZP byte Y	Zero-page address (hex, 1 byte) used to store the previous POTY value (e.g. `FE`)

Generated ASM shape (port 1, sprite 0, ZP `$FD`/`$FE`):

```
    ; CIA port select + settle
    LDA $DC00
    AND #$3F
    ORA #$40
    STA $DC00
    LDX #$67
wait:
    DEX
    BNE wait

    ; X axis — standard 1351-style 7-bit delta decode
    LDA $D419
    TAY
    SEC
    SBC $FD
    AND #$7F
    LDX #$00
    CMP #$40
    BCS xneg
    LSR A
    BEQ xdone
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
    JMP xdone
xneg:
    ORA #$C0
    CMP #$FF
    BEQ xdone
    SEC
    ROR
    DEX
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
xdone:
```

```
    ; Y axis — same decode, then inverted before apply
    LDA $D41A
    TAX
    SEC
    SBC $FE
    AND #$7F
    CMP #$40
    BCS yneg
    LSR A
    BEQ ydone
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
    JMP ydone
yneg:
    ORA #$C0
    CMP #$FF
    BEQ ydone
    SEC
    ROR
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
ydone:
```

Size: 142 bytes.

Expert mode syntax:

```
.mouse port, spriteNum, zpX, zpY
; example:
.mouse 2, 0, FD, FE
```

> **Important:** Before the first call, select the correct port with the CIA mux bits and initialise the zero-page bytes with the current POTX/POTY values to avoid a large jump on the first frame:
>
> ```
> ; port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
> ; port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
> LDA $D419 : LSR A : AND #$3F : STA $FD
> LDA $D41A : LSR A : AND #$3F : STA $FE
> ```

> **Recommended:** Poll the mouse once per frame, for example by placing `WAIT_RASTER` in the game loop before `MOUSE`, instead of hammering the SID POT registers in a tight loop.

> **Note:** The MOUSE macro selects the control port through CIA `$DC00` bits `7:6`: `%01` (`ORA #$40` after `AND #$3F`) for Control Port 1, `%10` (`ORA #$80` after `AND #$3F`) for Control Port 2. The lower

> six bits are preserved.

SPRITE_COL

Detects sprite collisions using the VIC-II hardware collision registers. Entirely **inline** — no JSR or label needed.

Field	Description
Sprite #	Sprite number 0–7 (which sprite's bit to check)
Collision type	`Sprite-Sprite ($D01E)` — collision with another sprite; `Sprite-Background ($D01F)` — collision with background graphics

Generated ASM (sprite 0, sprite–sprite):

```
    LDA $D01E       ; read sprite-sprite collision register (clears it!)
    AND #$01        ; isolate bit 0 (sprite 0)
                    ; A ≠ 0 → collision occurred
```

Size: 5 bytes.

> **Important:** Reading `$D01E` / `$D01F` **automatically clears the register**. Always read it exactly once per frame — use the result immediately with `BEQ`/`BNE`.

Typical usage:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    SPRITE_COL (sprite=0, type=sprite-sprite)
    BEQ no_hit          ; A = 0 → no collision
    LDA #$02
    STA $D020           ; red border = hit!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; light blue border = clear
    JMP gameloop
```

> **See also:** `collision-demo` sample — green ball (sprite #0) vs. red cross (sprite #1).

LOADFILE

Loads a file from a D64 disk at runtime using the C64 KERNAL routines SETNAM (`$FFBD`), SETLFS (`$FFBA`), and LOAD (`$FFD5`). Use this macro to load data, music, or additional code from disk while your program is running.

Field	Description
Filename	File name on the disk (max 16 chars, auto-uppercase; characters `,`, `"`, `/`, `\`, `:`, `*`, `?`, `<`, `>`, `|` are filtered)
Device	Device number 8–30 (default `8`)
Override address (optional)	Hex load address (e.g. `C000`). If set, the file is loaded to this address (`sec=0`, ignoring the PRG header). Leave empty to use the file's own 2-byte PRG header (`sec=1`).
Error label (optional)	If set, a `BCS` instruction is generated after JSR LOAD. If the KERNAL returns with carry set (error), execution jumps to this label.

Generated code structure:

```
    JMP skip_filename      ; jump over the inline filename
    .byte "DEMO-COLORS"    ; filename bytes (PETSCII, 11 chars)
skip_filename:
    LDA #11                ; filename length
    LDX #<fname            ; pointer lo
    LDY #>fname            ; pointer hi
    JSR $FFBD              ; SETNAM
    LDA #1                 ; logical file #1
    LDX #8                 ; device 8
    LDY #0                 ; sec=0 (override addr) or #1 (file's own addr)
    JSR $FFBA              ; SETLFS
    LDA #0                 ; LOAD command (not VERIFY)
    JSR $FFD5              ; LOAD
    BCS fail               ; (only if error label set)
```

Size: `3 + filename_length + 9 (SETNAM) + 9 (SETLFS) + (4 if override) + 5 (LOAD) + (2 if error label)` bytes. Minimum 27 bytes.

> **Important:** The filename is stored inline in the machine code right after a `JMP skip_filename`. The file name on the disk must be uppercase PETSCII — which matches plain ASCII uppercase letters (`A`–`Z`). The macro enforces this automatically.

> **Error handling:** The KERNAL sets the carry flag if LOAD fails (no drive, file not found, etc.). Without an error label, a failure leaves carry set and execution falls through to whatever follows — which is almost certainly wrong. Always provide an error label for production programs.

> **See also:** `loadfile-demo` sample — demonstrates loading `DEMO-COLORS.PRG` from a D64 with a BCS error branch and a visual error screen.

REU_CHECK

Detects whether a Commodore RAM Expansion Unit (REU) is present by writing test values to writable REU register `$DF04` and reading them back.

Generated code (34 bytes):

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF    ; Z=0 => REU present
BNE done
fail:
LDA #$FF
CMP #$FF    ; Z=1 => no REU
done:
```

> The macro normalizes the result so the following branch stays simple: `BNE` means REU present, `BEQ` means REU missing.

Result in flags:

- **Z = 0** (result ≠ 0) → REU present → use `BNE`
- **Z = 1** (result = 0) → no REU → use `BEQ`

No configurable fields — the macro generates the same code every time.

Typical usage:

```
REU_CHECK
BEQ no_reu        ; skip if REU not present
; ... REU code here ...
no_reu:
```

REU_STASH / REU_FETCH / REU_SWAP

Performs a block DMA transfer between C64 RAM and a Commodore REU using the REU's built-in DMA engine.

Macro	Direction	`$DF01` command
`REU_STASH`	C64 RAM → REU	`$90`
`REU_FETCH`	REU → C64 RAM	`$91`
`REU_SWAP`	C64 RAM ↔ REU	`$92`

Fields:

Field	Description	Example
C64 address	Source/dest in C64 RAM (hex)	`C000`
REU address	Source/dest in REU (hex, 16-bit)	`0000`
REU bank	REU memory bank (0–7)	`0`
Length	Number of bytes to transfer (hex, 16-bit)	`1000`

Generated code (40 bytes):

```
LDA #c64lo    STA $DF02    ; C64 address LO
LDA #c64hi    STA $DF03    ; C64 address HI
LDA #reuLo    STA $DF04    ; REU address LO
LDA #reuHi    STA $DF05    ; REU address HI
LDA #bank     STA $DF06    ; REU bank
LDA #lenLo    STA $DF07    ; length LO
LDA #lenHi    STA $DF08    ; length HI
LDA #$00      STA $DF09    ; address control (fixed)
LDA #$00      STA $DF0A    ; interrupt mask (fixed)
LDA #cmd      STA $DF01    ; execute DMA ($90/$91/$92 = stash/fetch/swap,
immediate)
```

> **Note:** Immediate DMA uses command values with bit 4 set (`$90/$91/$92`), which disables FF00-triggered mode. Writing to `$DF01` then starts the DMA immediately while the C64 CPU is halted during the transfer.

TURBO_SET

Sets the U64 (Ultimate-64) CPU turbo speed via register `$D031`.

Fields:

Field	Description	Range
Speed	CPU speed index	0 = 1 MHz … 7 ≈ 10 MHz … 15 ≈ 48 MHz
Badline	Badline emulation	Enabled (C64 compatible) / Disabled (turbo)

The speed byte is calculated as: `(speedIndex & 0x0F) | (badline_disabled ? 0x80 : 0x00)`.

Generated code (5 bytes):

```
A9 xx   LDA #speed_byte
8D 31 D0   STA $D031
```

Expert mode syntax:

```
.turbo_set 7,0    ; speed=7 (~10 MHz), badline enabled
.turbo_set 15,1   ; speed=15 (~48 MHz), badline disabled
```

> **Note:** This macro affects U64 hardware only. On a real C64 or other emulators this writes to `$D031` which may affect the CIA or be ignored.

SUPERCPU_DETECT

Detects whether a CMD SuperCPU accelerator is installed by reading its version register at `$D0B8` and comparing it to `$FF`.

Generated code (5 bytes):

```
AD B8 D0   LDA $D0B8
C9 FF      CMP #$FF
```

Result in flags:

- **Z = 0** → SuperCPU present → use `BNE`
- **Z = 1** → SuperCPU not found → use `BEQ`

No configurable fields.

Typical usage:

```
SUPERCPU_DETECT
BEQ no_scpu       ; skip if SuperCPU not present
; ... SuperCPU turbo code here ...
no_scpu:
```

TURBO_ENABLE

Enables or disables CMD SuperCPU turbo mode by writing to the SuperCPU control registers.

Mode	Register	Effect
Enable	`$D07A`	Engage turbo (up to 20 MHz with SuperCPU)
Disable	`$D07B`	Return to 1 MHz compatibility mode

Generated code (5 bytes):

```
A9 00         LDA #$00
8D 7A D0      STA $D07A    ; (or $D07B for disable)
```

Expert mode syntax:

```
.turbo_enable on
.turbo_enable off
```

> **Note:** Call `SUPERCPU_DETECT` first and branch around this macro if the SuperCPU is not present.

---

10. Debugger Integration

The app supports **RetroDebugger** as the external C64 debugger. It receives breakpoints, symbols, and autostart flags generated from the assembled program.

RetroDebugger

RetroDebugger is a cross-platform Commodore 64 debugger with breakpoint support, memory inspection, and label-aware disassembly.

Setup: Open **Settings → Configure RetroDebugger executable** and point it to the `RetroDebugger` binary.

Launch: Click **Debug (RetroDebugger)** in the toolbar. The app will:

1. Assemble the program to a `.prg` file in a temporary directory.
2. Write a **breakpoints file** (`breakpoints.txt`) — one `break $ADDR` per flagged block.
3. Write a **symbols file** (`symbols.txt`) in Vice/RetroDebugger label format (`al C:addr .name`). All LABEL and CONST blocks are included.
4. Launch RetroDebugger with:

```
RetroDebugger -prg <file.prg> -breakpoints <breakpoints.txt> -symbols
<symbols.txt> [flags]
```

Breakpoint Blocks

Click the breakpoint icon (●) on any instruction block to toggle it as a breakpoint. Breakpointed blocks are highlighted in red. Their addresses are written to the breakpoints file on every debugger launch.

Debugger Flags (Options Tab)

Toggle	Flag	Effect
`-jmp` ON	`-jmp $ADDR`	Jump directly to the program start address after loading
`-unpause` ON	`-unpause`	Unpause the debugger immediately on load
`-wait` ON	`-wait <ms>`	Wait `<ms>` milliseconds before unpausing — 500 ms or 1000 ms

> **Tip:** For most programs, enable `-jmp` and `-unpause` for instant autostart. Use `-wait 500` or `-wait 1000` when your program sets up IRQs or SID music that needs time to initialize before the first raster.

---

11. Knowledge Base Links

Quick reference links available in the app under **Knowledge Base**:

Resource	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html
VIC-II Article	https://www.cebix.net/VIC-Article.txt
C64 Codebase	https://codebase.c64.org/
The Turbo Assembler	https://turbo.style64.org/
RetroDebugger	https://github.com/slajerek/RetroDebugger/

---

12. ASM Import Workflow

The Import ASM dialog now includes quality-of-life safeguards for iterative fixes:

- **Line numbers in the import editor** are synchronized with scroll position.
- **Inline error list** is shown at the bottom of the same dialog when import fails.
- **Source preservation on failure** restores your pasted source automatically, so you can correct and retry without re-pasting.

This keeps the import-debug cycle inside one dialog: paste, import, inspect errors, fix, retry.

---

13. D64 Export & Run

Version 1.5.1 adds the ability to package your program (and additional data files) into a C64 D64 disk image and launch it in VICE — or export the disk image for use elsewhere.

Split Run button

The toolbar **Run** button has been replaced by a **split button**:

Part	Action
▶ **Run** (main)	Executes the currently selected run mode
▾ (arrow)	Opens the mode selector

Available run modes:

Mode	Description
Run as PRG	Assemble to a temporary `.prg` and launch VICE directly. Classic behaviour.

Mode	Description
Run via D64	Assemble, build a `.d64` disk image (using c1541), add any configured extra files, then launch VICE from the disk. Use this whenever your program loads files at runtime (e.g. with the LOADFILE macro).
Run on hardware	Assemble to PRG and send it to a **1541 Ultimate / Ultimate 64** device over the local network. See Section 13.

The selected mode is saved between sessions.

Export to D64 dialog

Open via the **Save PRG** ▾ dropdown → **Export to D64**. The dialog lets you:

1. Set the **disk name** (max 16 chars) and **program name** — these are the names that appear in the C64 disk directory.
2. **Add extra files** — click **+** to pick any binary file (`.prg`, `.bin`, `.sid`, etc.). For each extra:
   - **Name** — how it appears in the D64 directory (max 16 chars, auto-uppercase).
   - **Load address (optional)** — if provided, a 2-byte PRG header is prepended. Leave empty to write raw bytes with no header.
3. Click **Export** to generate the `.d64` file using VICE's `c1541` tool.

D64 metadata in projects

The disk name, program name, and extra file list are saved inside the project JSON (under the `d64` key). When you reload the project or a sample that includes D64 metadata, the extras are restored automatically — no need to re-add them each time.

The **loadfile-demo** sample comes pre-configured with `DEMO-COLORS.PRG` as an extra file. Select it, open **Run via D64**, and click **Run** to see the full load flow in action.

> **Requirement:** D64 export and Run via D64 both require VICE (`c1541`) to be configured in Hardware Settings.

---

14. Hardware Settings

Open via **Settings → Hardware Settings...** in the toolbar menu. All external hardware paths and network configuration live here.

VICE Emulator

Setting	Description
Select VICE	Browse to the `x64sc` (or `x64`) VICE executable
Status	Shows whether the executable path is valid and accessible

VICE is required for **Run as PRG**, **Run via D64**, and **Export to D64**.

Retro Debugger

Setting	Description
Select RetroDebugger	Browse to the `RetroDebugger` binary
Status	Shows whether the path is valid

See Section 9 for full debugger documentation.

C64 Ultimate / 1541 Ultimate

Run assembled PRGs directly on real hardware over the local network using the Ultimate REST API.

Setting	Description
Host (IP)	IP address of the device (e.g. `192.168.1.100`)
Password	Optional — if the device requires authentication
Test connection	Sends a test request to `/v3/runners/info`; shows OK or error

Workflow:

1. Connect the 1541 Ultimate / Ultimate 64 to your local network.
2. Enter its IP address (and password if set) in Hardware Settings.
3. Select **Run on hardware** from the split run menu.
4. Click ▶ **Run** — the PRG is compiled and sent to the device via HTTP POST to `/v3/runners/prg`. The device loads and runs it on the C64 immediately.

> **Tip:** No USB cable or driver needed — the REST API is built into the Ultimate firmware. Your computer and the device must be on the same local network.
